

Source-Controlled Testing, Health, and Security Evidence

Document field	Value
Document owner	FortVault
Classification	Confidential
Status	Source-reviewed implementation evidence
Date	August 28, 2026

Scope and Evidence Limits

This document summarizes behavior evidenced by the reviewed source repositories. It does not prove that a given test ran in production, that a cloud control is enabled, or that operational logs, backups, TLS, or endpoint controls are effective.

Repositories reviewed for this document:

- fortvault-backend
- fortvault-processing
- fortvault-mpc
- fortvault-health-audit

Automated Testing

Component	Source-controlled test evidence	Standard commands declared in source
Backend	Jest unit tests, end-to-end test configuration, health-check tests, and TypeScript build/lint scripts	npm test, npm run test:cov, npm run test:e2e, npm run lint, npm run build
Custody Processing	Jest unit tests, end-to-end test configuration, and health-check tests	npm test, npm run test:e2e, npm run lint, npm run build
MPC	Go tests covering configuration, authorization, health, transport, key generation, encrypted key storage, signing sessions, Bitcoin, Tron, and Ethereum behavior	go test ./...
Health Audit	Automated tests for the read-only audit client and manifest validation	npm test

The exact test selection and release acceptance criteria should be recorded in the FortVault change-management procedure. A test command in a repository is not proof that all tests passed for a particular production release; attach CI output or signed release evidence for that purpose.

Health Checks

Component health endpoints

Backend, Custody Processing, and MPC implement a liveness/readiness/configuration health pattern:

- GET /health/live checks that the service can respond.

- `GET /health/ready` returns an unhealthy/degraded result when required dependencies are unavailable.
- `GET /health/config` provides a sanitized configuration view for the health audit and is protected by a dedicated audit key.

The health-check tests assert that the configuration response does not expose passwords, credentials, tokens, RPC URLs, database URLs, or health-audit secrets.

Read-only health audit

`fortvault-health-audit` is a source-controlled, read-only audit client. It checks the health endpoints for declared services and MPC clusters, validates expected partner/policy configuration, and can verify Redis-stream and consumer-group consistency.

The audit manifest is designed to contain service URLs and expected configuration only. The corresponding audit key is supplied at runtime, rather than committed to the manifest.

Logging Evidence

The Backend includes a Winston-based logging module with JSON log formatting and console output. The configured production level is `info`; development uses a more verbose level. The source also supports error and combined file transports outside production.

Application code and the health-check tests demonstrate an intent to avoid exposing secrets in health responses. This is not sufficient evidence of centralized production log retention, access control, alerting, or a completed review of every logging call. Those items remain operational evidence.

Application-Level Secret and Encryption Evidence

Area	Source-controlled evidence	Evidence limit
Secret loading	Custody Processing includes an AWS Secrets Manager loader used by its bootstrap CLI.	Confirm the actual production secret store, IAM policy, rotation, and access history outside Git.
MPC key-share protection	MPC configuration includes an encryption key for key-share material at rest; generated node configuration identifies that value as key-share encryption material.	Confirm production key management, storage encryption, node isolation, backup, and recovery procedures outside Git.
Health data minimization	Health configuration responses are authenticated and tested to exclude sensitive runtime values.	Confirm health endpoints are network-restricted and the audit key is managed/rotated operationally.
Transport encryption	The source uses HTTPS/RPC client configuration where external services require it.	TLS termination, minimum TLS version, cipher policy, certificates, and internal service transport settings are deployment controls and must be evidenced outside Git.

Source References

Area	Repository path
Backend scripts and test configuration	fortvault-backend/package.json
Backend health implementation and tests	fortvault-backend/src/modules/health-checker/
Backend logging implementation	fortvault-backend/src/shared/logger/
Processing scripts and test configuration	fortvault-processing/package.json
Processing health implementation and tests	fortvault-processing/src/modules/health-checker/
Processing AWS secret loader	fortvault-processing/src/shared/config/aws-secrets.loader.ts
MPC health implementation and tests	fortvault-mpc/app/health.go, fortvault-mpc/app/health_test.go
MPC configuration and encrypted-key tests	fortvault-mpc/configs/, fortvault-mpc/repository/db/encrypted_keys_test.go
Health audit README, implementation, tests, and example manifest	fortvault-health-audit/README.md, src/audit.mjs, test/audit.test.mjs, environments/example.json

Required Companion Operational Evidence

See 02-operational-evidence-checklist.md for the production artifacts required to complete the ISO response.